

UNIVERSIDAD CENTRAL DEL ECUADOR

Facultad de Ingeniería y Ciencias Aplicadas

Carrera de Computación

**Predicción del metajuego en Clash Royale
usando inteligencia artificial con datos de
2023-2026**

Integrantes:

Cajas Molina Robinson Stalin

Baraja Simbaña Cristian Paul

Periodo académico

2026-2026

Tabla de contenidos

Resumen	5
1. Planteamiento del Problema y Objetivos	7
1.1. Planteamiento del problema	7
1.2. Pregunta de investigación	8
1.3. Hipótesis	8
1.4. Objetivo general	8
1.5. Objetivos específicos	8
1.6. Justificación	9
1.7. Marco conceptual	9
1.8. Arquetipos considerados	10
1.9. Marco Metodológico	11
2. Marco Teórico	13
2.1. Teoría de Juegos y Decisiones Estratégicas	13
2.2. Fundamentos del Aprendizaje Automático (Machine Learning)	14
2.3. Series Temporales y Pronóstico	14
2.4. Redes Neuronales Recurrentes (RNN)	15
2.4.1. Long Short-Term Memory (LSTM)	15
2.4.2. Gated Recurrent Unit (GRU)	16
2.5. Modelos de Regresión y Complejidad del Modelo	16
2.6. Métodos de Ensamble	16

2.6.1.	Bagging (Bootstrap Aggregating)	17
2.6.2.	Gradient Boosting	17
2.7.	Modelos de Pronóstico Temporal	17
2.7.1.	Facebook Prophet	17
2.8.	Métricas de Evaluación de Modelos	18
2.9.	Análisis de eSports y Minería de Datos	18
2.10.	Tecnologías e Innovaciones Incorporadas en la Creación del Aplicativo	19
2.10.1.	Formato ONNX y Motor de Inferencia ONNX Runtime	19
2.10.2.	Arquitectura Web Frontend: Single Page Application (SPA) y Tecnologías UI	20
2.10.3.	Arquitectura Backend, Protocolo HTTP y API REST	20
2.10.4.	Infraestructura de Despliegue Serverless (Vercel Edge Platform)	21
2.10.5.	Ecosistema de Ingestión, Preprocesamiento y Data Science en Python	21
2.10.6.	Arquitectura de Ensamble Híbrido Multiescala (LSTM + LightGBM)	22
2.10.7.	Motor de Simulación Estocástica Monte Carlo	22
3.	Selección de Modelos Candidatos y Justificación Metodológica	23
3.1.	Modelos Candidatos Evaluados	23
3.1.1.	Random Forest (Regresor)	23
3.1.2.	XGBoost (Extreme Gradient Boosting)	24
3.1.3.	LightGBM (Light Gradient Boosting Machine)	24
3.1.4.	LSTM (Long Short-Term Memory)	24
3.1.5.	GRU (Gated Recurrent Unit)	25
3.1.6.	Prophet (Facebook)	25
3.2.	Tecnologías e Innovaciones Incorporadas en la Creación del Aplicativo	26
3.2.1.	Formato ONNX y Motor de Inferencia ONNX Runtime	26
3.2.2.	Arquitectura de Ensamble Híbrido Multiescala (LSTM + LightGBM)	26
3.2.3.	Motor de Simulación Estocástica Monte Carlo	27
3.2.4.	Stack Tecnológico Web REST y Despliegue Serverless	27
3.2.5.	Ecosistema de Ingestión y Analítica en Python	27

4. Proceso Experimental y Validación Temporal	28
4.1. División del Conjunto de Datos (Train-Validation-Test)	28
4.2. Optimización de Hiperparámetros y Prevención del Sobreajuste	29
4.2.1. Modelos de Ensamble de Árboles (RF, XGBoost, LightGBM)	29
4.2.2. Modelos Neuronales Recurrentes (LSTM y GRU)	29
4.2.3. Modelos Estadísticos (Prophet)	29
5. Análisis Estadístico del Conjunto de Datos	31
5.1. Estructura y Variables Disponibles	31
5.1.1. Datos Temporales	31
5.1.2. Datos Categóricos y de Composición de Mazos	32
5.1.3. Datos Numéricos de Popularidad y Rendimiento	32
5.2. Resultados del Análisis Global y Meso	32
5.2.1. Comparativa de Parámetros Globales (Ganadores vs. Perdedores)	32
5.2.2. Distribución de Popularidad y Desempeño por Arquetipo (Meso)	33
6. Análisis Avanzado a Nivel Micro y Correlaciones de Victoria	35
6.1. Composición de Rarezas (Mazos Ganadores vs. Mazos Perdedores)	35
6.2. Análisis de Sinergias (Combos de Cartas con Mayor Éxito)	36
6.3. Importancia de las Características en la Predicción de Victoria	38
6.4. Prueba de Significancia Estadística de la Hipótesis	39
7. Evaluación Comparativa de los Modelos	41
7.1. Análisis del Desempeño y Curvas de Error	42
7.1.1. Curva de Predicción Temporal del Metajuego	42
7.2. Prueba de Significancia Estadística (t de Student)	43
8. Modelo Final, Justificación Científica y Arquitectura	45
8.1. Selección y Justificación Científica del Modelo Final	45
8.1.1. Justificación de la Red LSTM	45
8.1.2. Justificación de LightGBM (como Clasificador de Combates Complementario)	46

8.1.3. Módulos del Sistema:	46
9. Construcción del Aplicativo	47
9.1. Metodología de Desarrollo	47
9.2. Arquitectura del Sistema	48
9.2.1. Diagrama de Componentes	48
9.2.2. Flujo de Datos: Predicción de Mazo	50
9.2.3. Flujo de Datos: Simulación de Parches	51
9.3. Justificación de Tecnologías	52
9.3.1. Frontend: React + TypeScript	52
9.3.2. Backend: Express.js + ONNX Runtime	52
9.3.3. Despliegue: Vercel	53
9.4. Pipeline de Entrenamiento y Exportación de Modelos	53
9.5. Estructura y Módulos del Aplicativo	54
9.5.1. Componentes de la Interfaz (Frontend)	54
9.5.2. Controlador API REST (Backend)	54
9.6. Métricas de Rendimiento	55
9.7. Despliegue y Escalabilidad	56
10. Conclusiones e Investigación de Campo	57
Referencias	58

Resumen

El presente trabajo desarrolla una investigación aplicada sobre la predicción del metajuego en *Clash Royale* usando inteligencia artificial y minería de datos con enfoque en el periodo 2023-2026. El metajuego se entiende como la distribución dinámica de cartas, mazos, arquetipos y estrategias dominantes dentro del entorno competitivo. En este videojuego, el meta cambia por la interacción entre parches de balance, incorporación de evoluciones de cartas, comportamiento de la comunidad, uso de arquetipos populares y resultados competitivos observados en partidas.

La investigación recolecta, procesa y estructura datos relacionados con rendimiento, popularidad y uso de cartas y mazos. A partir de estos datos se construyen variables como tasa de uso, tasa de victoria, costo promedio de elixir, nivel total de cartas, distribución de rarezas, sinergias entre cartas, condiciones de victoria y clasificación por arquetipos. Se planteó como hipótesis que los modelos secuenciales de aprendizaje automático (LSTM, GRU) obtendrían un error de predicción menor que los modelos estáticos (Random Forest, XGBoost, LightGBM) al predecir la evolución temporal del metajuego, y que las variables de habilidad y progresión del jugador (copas iniciales y nivel de cartas) tendrían mayor peso predictivo que la rareza de las cartas.

El estudio aplica el método analítico, descomponiendo el metajuego en sus componentes fundamentales (cartas, mazos, arquetipos, métricas de rendimiento) para su posterior reconstrucción mediante modelos predictivos. Se entrenaron y compararon seis modelos: Random Forest, XGBoost, LightGBM, LSTM, GRU y Prophet, con el objetivo de estimar cambios temporales del metajuego y anticipar tendencias futuras.

Los resultados experimentales muestran que los modelos de aprendizaje automático pueden capturar patrones de evolución del meta cuando las partidas se agrupan en ventanas temporales y se

representan mediante arquetipos. En la comparación de modelos, LSTM obtuvo el menor error global con MAE de 0.006103 y RMSE de 0.007541, seguido de Random Forest y LightGBM con desempeños muy cercanos. Una prueba t de Student pareada determinó que la diferencia entre LSTM y Random Forest no es estadísticamente significativa ($p = 0.2646$), indicando que ambos modelos ofrecen un rendimiento comparable. Además, el análisis de importancia de variables evidenció que las diferencias de copas iniciales (0.1899) y nivel de cartas (0.1374) tienen una importancia 84 % mayor que las variables de rareza, confirmando que la progresión del jugador pesa más que la composición compositiva del mazo. Se concluye que la inteligencia artificial es una herramienta viable para anticipar tendencias estratégicas en *Clash Royale* y apoyar el análisis competitivo en eSports.

Palabras clave: Clash Royale, metajuego, inteligencia artificial, aprendizaje automático, series temporales, LSTM, LightGBM, minería de datos, eSports

Capítulo 1

Planteamiento del Problema y Objetivos

1.1. Planteamiento del problema

Clash Royale es un videojuego competitivo de estrategia en tiempo real en el que dos jugadores se enfrentan utilizando mazos de ocho cartas. Cada carta posee atributos específicos, como costo de elixir, daño, puntos de vida, velocidad, rareza, tipo de unidad y función táctica. La combinación de estas cartas produce mazos con identidades estratégicas distintas, conocidos como arquetipos.

El metajuego, o *meta*, representa el conjunto de cartas, mazos y estrategias que dominan el entorno competitivo durante un periodo determinado. Entre 2023 y 2026, el metajuego de *Clash Royale* se volvió especialmente dinámico debido a la aparición de evoluciones de cartas, ajustes de balance, cambios en progresión, temporadas competitivas y respuestas rápidas de la comunidad. Supercell introdujo las evoluciones de cartas en 2023, agregando nuevas habilidades a cartas existentes y modificando profundamente la construcción de mazos. En 2026, las actualizaciones continuaron incorporando nuevas evoluciones, héroes, modos de juego y ajustes de progresión.

El problema de investigación consiste en determinar si es posible predecir cambios y tendencias futuras del metajuego mediante inteligencia artificial. Esta tarea es compleja porque el rendimiento de una carta no depende solo de sus estadísticas individuales, sino también de sus sinergias, counters, popularidad, habilidad del jugador, frecuencia de uso y contexto competitivo.

1.2. Pregunta de investigación

¿Cómo pueden utilizarse técnicas de inteligencia artificial y minería de datos para predecir cambios y tendencias futuras del metajuego de *Clash Royale* durante el periodo 2023-2026?

1.3. Hipótesis

Se plantea que los modelos de aprendizaje automático con capacidad secuencial (LSTM, GRU) obtendrán un error de predicción menor que los modelos estáticos (Random Forest, XGBoost, LightGBM) al predecir la evolución temporal del metajuego, debido a su capacidad de capturar dependencias acumulativas de largo plazo. Asimismo, se hipotetiza que la diferencia en el nivel acumulado de cartas y las copas iniciales del jugador son los factores más determinantes para predecir el resultado de una partida, superando la influencia de la composición del mazo o la rareza de las cartas.

1.4. Objetivo general

Desarrollar un modelo de inteligencia artificial capaz de predecir cambios y tendencias futuras en el metajuego de *Clash Royale*, utilizando datos relacionados con el rendimiento, popularidad y uso de cartas y mazos durante el periodo 2023-2026.

1.5. Objetivos específicos

A continuación se detallan las metas a cumplir:

1. Recolectar y procesar datos relacionados con el rendimiento, popularidad y uso de cartas y mazos en *Clash Royale* durante el período 2023-2026.
2. Analizar patrones temporales y variaciones del metajuego utilizando técnicas de IA.
3. Entrenar y evaluar modelos de inteligencia artificial para predecir cambios y tendencias futuras en el metajuego de *Clash Royale*.

4. Comparar el rendimiento de distintos modelos predictivos mediante métricas de precisión y exactitud.
5. Identificar los factores que más influyen en la evolución del metajuego, como balance de cartas, frecuencia de uso y efectividad de arquetipos.

1.6. Justificación

La predicción del metajuego es relevante porque permite comprender cómo evoluciona un sistema competitivo complejo. En *Clash Royale*, un cambio de balance sobre una carta puede modificar no solo su tasa de uso, sino también la viabilidad de mazos completos y de arquetipos relacionados. Por ejemplo, una mejora en una condición de victoria puede aumentar su popularidad y, al mismo tiempo, elevar la presencia de cartas diseñadas para contrarrestarla.

Desde la ciencia de datos, el metajuego puede representarse como una serie temporal multivariante. Cada ventana temporal contiene señales de popularidad, rendimiento y adopción estratégica. Desde la inteligencia artificial, este problema puede abordarse con modelos tabulares, ensambles de árboles, redes recurrentes y modelos de pronóstico. Desde los eSports analytics, la predicción del meta permite apoyar decisiones de entrenamiento, selección de mazos, preparación de counters y análisis de torneos.

1.7. Marco conceptual

El análisis se organiza alrededor de cinco conceptos:

Concepto	Definición operativa
Carta	Unidad básica del juego con atributos, rareza, costo y función táctica
Mazo	Conjunto de ocho cartas usado por un jugador
Arquetipo	Categoría estratégica del mazo según condición de victoria y estilo de juego

Concepto	Definición operativa
Tasa de uso	Frecuencia relativa con la que aparece una carta, mazo o arquetipo
Tasa de victoria	Porcentaje de victorias obtenido por una carta, mazo o arquetipo

Fuente: Elaboración propia basada en las mecánicas de juego de Supercell (2023-2026).

1.8. Arquetipos considerados

Para reducir la complejidad combinatoria, los mazos se agrupan en arquetipos:

Arquetipo	Descripción
Beatdown	Mazos pesados que acumulan tropas detrás de tanques o condiciones de victoria resistentes
Cycle/Control rápido	Mazos de bajo costo que rotan rápido y presionan de forma constante
Asedio	Mazos basados en estructuras ofensivas como Ballesta o Mortero
Bridge Spam	Mazos que presionan en el puente para castigar gasto ineficiente de elixir
Control lento	Mazos defensivos que buscan contraataques de alto valor
Híbridos	Mazos que combinan varias condiciones de victoria o no encajan en una sola categoría

Fuente: Clasificación y elaboración propia basada en el procesamiento del conjunto de datos histórico de partidas de Clash Royale (2023-2026).

Esta clasificación permite transformar millones de combinaciones posibles en grupos interpretables para el análisis predictivo.

1.9. Marco Metodológico

La presente investigación adopta el **método analítico** como estrategia principal de investigación. Este método consiste en descomponer un fenómeno complejo en sus componentes fundamentales para comprender su funcionamiento interno, y posteriormente reconstruirlo mediante síntesis intelectual.

El proceso metodológico se desarrolló en las siguientes fases:

1. **Definición del objeto de estudio:** Se delimitó el fenómeno del metajuego de *Clash Royale* como un sistema dinámico compuesto por cartas, mazos, arquetipos y jugadores que interactúan entre sí.
2. **Formulación de hipótesis:** Se planteó que los modelos secuenciales de inteligencia artificial (LSTM, GRU) serían más efectivos para predecir la evolución temporal del metajuego que los modelos estáticos, y que las variables de habilidad y progresión del jugador tendrían mayor peso predictivo que la rareza de las cartas.
3. **Descomposición:** El fenómeno se separó en sus elementos fundamentales: cartas individuales (con atributos como costo, daño, rareza), mazos (combinaciones de ocho cartas), arquetipos (categorías estratégicas) y métricas de rendimiento (tasa de uso, tasa de victoria).
4. **Clasificación y organización:** Los datos obtenidos se ordenaron mediante ventanas temporales de 1,000 partidas, clasificando los mazos en arquetipos y calculando estadísticas descriptivas globales, meso y micro.
5. **Recomposición:** Se entrenaron y compararon seis modelos predictivos (Random Forest, XGBoost, LightGBM, LSTM, GRU y Prophet) para reconstruir la dinámica del metajuego a partir de las partes analizadas, seleccionando un ensamble híbrido LSTM + LightGBM como sistema final.
6. **Validación:** Se aplicaron pruebas de significancia estadística (t de Student) para verificar

las hipótesis planteadas y confirmar que las diferencias observadas entre modelos y factores explicativos son coherentes con la realidad del fenómeno estudiado.

Capítulo 2

Marco Teórico

El presente marco teórico fundamenta los constructos conceptuales, estadísticos y computacionales que sustentan la investigación, integrando disciplinas como la **Teoría de Juegos**, la **Ciencia de Datos**, el **Aprendizaje Automático** y el **Análisis de Series Temporales**.

2.1. Teoría de Juegos y Decisiones Estratégicas

La **Teoría de Juegos** se define como el análisis riguroso y sistemático de situaciones de conflicto y cooperación en las que interactúan individuos racionales (Cerdá Tena et al., 2004). En estos escenarios, cada jugador busca maximizar su **función de utilidad**, pero el resultado final no depende únicamente de sus propias acciones, sino de las decisiones tomadas por los demás participantes (Cerdá Tena et al., 2004).

Un concepto central es el **Equilibrio de Nash**, un perfil de estrategias en el cual la elección de cada jugador es una respuesta óptima a las estrategias del resto; de este modo, ningún individuo tiene incentivos para desviarse de su decisión de manera unilateral (Cerdá Tena et al., 2004). Este marco permite predecir comportamientos en entornos competitivos donde las reglas son conocidas por todos los agentes (Cerdá Tena et al., 2004).

En el contexto de los videojuegos competitivos como *Clash Royale*, la Teoría de Juegos permite modelar las interacciones estratégicas entre jugadores que eligen mazos y arquetipos buscando

maximizar su probabilidad de victoria, considerando las respuestas esperadas de sus oponentes.

2.2. Fundamentos del Aprendizaje Automático (Machine Learning)

La **Ciencia de Datos** se fundamenta en principios que guían la extracción de conocimiento a partir de los datos (Fernández Genaro, 2023). Dentro de esta disciplina, el **Aprendizaje Automático** confiere a los sistemas la capacidad de aprender de la experiencia para mejorar su desempeño en tareas específicas sin ser programados explícitamente (Fernández Genaro, 2023; Russell, 2018).

Según la naturaleza de los datos, los sistemas se clasifican en:

- **Aprendizaje Supervisado:** El modelo se entrena con datos etiquetados (entradas y salidas deseadas) para aprender una función de mapeo que permita realizar tareas de **clasificación** o **regresión** (Fernández Genaro, 2023; Russell, 2018).
- **Aprendizaje No Supervisado:** El algoritmo identifica estructuras ocultas en datos no etiquetados, destacando técnicas como el **Clustering** (agrupamiento de observaciones similares) y la **reducción de dimensionalidad** (Fernández Genaro, 2023; Russell, 2018).

2.3. Series Temporales y Pronóstico

Una **serie temporal** es una secuencia de observaciones numéricas ordenadas cronológicamente, donde cada valor depende de los anteriores y/o de factores externos. El **análisis de series temporales** estudia los patrones de tendencia, estacionalidad y ruido para generar pronósticos futuros (Hyndman & Athanasopoulos, 2021).

Los componentes fundamentales de una serie temporal son:

- **Tendencia (Trend):** Dirección a largo plazo de los datos (creciente, decreciente o estable).
- **Estacionalidad (Seasonality):** Fluctuaciones periódicas que se repiten en intervalos regulares.
- **Ruido (Noise):** Variación aleatoria no explicada por los componentes anteriores.

En el metajuego de *Clash Royale*, las series temporales representan la evolución de las tasas de uso y victoria de los arquetipos a lo largo del tiempo, donde cada ventana temporal contiene las proporciones de cada estrategia competitiva (Hyndman & Athanasopoulos, 2021).

2.4. Redes Neuronales Recurrentes (RNN)

Las **Redes Neuronales Recurrentes (RNN)** son una clase de redes neuronales diseñadas para procesar secuencias de datos, donde la información de pasos anteriores se propaga a través de un estado oculto (*hidden state*) que funciona como memoria de corto plazo (Russell, 2018).

Sin embargo, las RNN estándar sufren el problema del **desvanecimiento del gradiente** (*vanishing gradient*): durante el entrenamiento por retropropagación a través del tiempo (*Backpropagation Through Time*, BPTT), los gradientes se reducen exponencialmente a medida que se propagan hacia atrás, dificultando el aprendizaje de dependencias a largo plazo.

2.4.1. Long Short-Term Memory (LSTM)

Las **LSTM** fueron propuestas por Hochreiter y Schmidhuber en 1997 para resolver el problema del desvanecimiento del gradiente. Una celda LSTM utiliza tres compuertas (*gates*) para controlar el flujo de información:

- **Compuerta de olvido (*forget gate*):** Decide qué información descartar del estado de celda anterior.
- **Compuerta de entrada (*input gate*):** Decide qué nueva información almacenar en el estado de celda.
- **Compuerta de salida (*output gate*):** Decide qué parte del estado de celda outputear como estado oculto.

Esta arquitectura permite a las LSTMs capturar dependencias a largo plazo en secuencias de datos, siendo superiores a las RNN estándar en tareas como reconocimiento de voz, traducción automática y pronóstico de series temporales.

2.4.2. Gated Recurrent Unit (GRU)

Las **GRU** fueron propuestas como una simplificación de las LSTM, combinando las compuertas de olvido y entrada en una única compuerta de actualización, y eliminando el estado de celda separado.

Utiliza dos compuertas:

- **Compuerta de reinicio (*reset gate*):** Controla cuánta información anterior ignorar al calcular el nuevo estado candidato.
- **Compuerta de actualización (*update gate*):** Decide cuánto del estado anterior conservar versus cuánto actualizar con el nuevo candidato.

Las GRU tienen menos parámetros que las LSTM, lo que las hace más rápidas de entrenar manteniendo una capacidad predictiva comparable en muchos problemas.

2.5. Modelos de Regresión y Complejidad del Modelo

Para la predicción de variables continuas, se emplean modelos de **regresión lineal**, que calculan una suma pesada de las características de entrada junto con una constante de tendencia (Russell, 2018). En conjuntos de datos complejos, se utiliza la **regresión polinomial** para ajustar funciones no lineales (Russell, 2018).

Un desafío crítico es el **sobreajuste (*overfitting*)**, que ocurre cuando un modelo es demasiado complejo para la cantidad de datos disponibles, logrando un excelente desempeño en el entrenamiento pero fallando al generalizar en datos nuevos (Russell, 2018). Para mitigar esto, se utilizan técnicas de **regularización** como la **Regresión Contraída (Ridge)** o la **Regresión Lasso**, que penalizan la complejidad del modelo añadiendo un peso a la función de costo (Russell, 2018).

2.6. Métodos de Ensamble

Los **métodos ensemble** combinan múltiples modelos predictivos para generar una predicción final más precisa y robusta que la de un experto individual (Fernández Genaro, 2023). Entre las técnicas más utilizadas se encuentran:

2.6.1. Bagging (Bootstrap Aggregating)

Entrena múltiples modelos independientes en subconjuntos de datos aleatorios con reemplazo, promediando sus predicciones para reducir la varianza. El ejemplo más representativo es **Random Forest** propuesto por Breiman en 2001 (Breiman, 2001; Fernández Genaro, 2023).

2.6.2. Gradient Boosting

Construye modelos de forma secuencial, donde cada nuevo árbol corrige los errores residuales del anterior mediante optimización por descenso de gradiente (Fernández Genaro, 2023). Las implementaciones más destacadas son:

- **XGBoost** (*eXtreme Gradient Boosting*): Sistema escalable de gradient boosting propuesto por Chen y Guestrin en 2016, que incorpora regularización L1/L2 y manejo eficiente de datos dispersos.
- **LightGBM** (*Light Gradient Boosting Machine*): Algoritmo propuesto por Ke et al. en 2017 que utiliza muestreo unidireccional basado en gradiente (GOSS) y empaquetado de características exclusivas (EFB) para acelerar el entrenamiento hasta 20 veces respecto a XGBoost sin perder precisión.

2.7. Modelos de Pronóstico Temporal

2.7.1. Facebook Prophet

Prophet es un modelo de pronóstico temporal desarrollado por Facebook, basado en un modelo aditivo que descompone la serie en componentes de tendencia (lineal o no lineal), estacionalidades (diaria, semanal, anual) y efectos de días festivos. Es robusto ante valores atípicos y cambios estructurales, y está diseñado para series temporales con patrones estacionales fuertes y varios periodos de datos históricos. Su función principal es:

$$y(t) = g(t) + s(t) + h(t) + \epsilon_t$$

Donde $g(t)$ representa la tendencia, $s(t)$ la estacionalidad, $h(t)$ los efectos de festivos y ϵ_t el ruido (Facebook/Meta Prophet contributors, 2025).

2.8. Métricas de Evaluación de Modelos

Para comparar el rendimiento de los modelos predictivos se utilizan las siguientes métricas estándar (Hyndman & Athanasopoulos, 2021):

- **Error Absoluto Medio (MAE):** Promedio de los errores absolutos entre las predicciones y los valores reales. Mide la magnitud promedio del error.

$$MAE = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i|$$

- **Raíz del Error Cuadrático Medio (RMSE):** Raíz cuadrada del promedio de los errores al cuadrado. Penaliza más los errores grandes.

$$RMSE = \sqrt{\frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2}$$

- **Coefficiente de Determinación (R^2):** Proporción de la varianza total explicada por el modelo. Varía entre 0 y 1, donde 1 indica ajuste perfecto.

$$R^2 = 1 - \frac{\sum_{i=1}^n (y_i - \hat{y}_i)^2}{\sum_{i=1}^n (y_i - \bar{y})^2}$$

2.9. Análisis de eSports y Minería de Datos

El **análisis de eSports** (*eSports Analytics*) aplica técnicas de ciencia de datos y aprendizaje automático para extraer conocimiento de los datos generados por videojuegos competitivos. En juegos como *Clash Royale*, los datos disponibles incluyen composición de mazos, niveles de cartas, tasas de uso, resultados de partidas y métricas de rendimiento.

La **minería de datos** consiste en aplicar algoritmos de descubrimiento de conocimiento sobre gran-

des volúmenes de datos para identificar patrones, correlaciones y tendencias útiles para la toma de decisiones (Russell, 2018). En este estudio, la minería de datos se utiliza para clasificar arquetipos, identificar sinergias entre cartas y determinar los factores que más influyen en la victoria.

2.10. Tecnologías e Innovaciones Incorporadas en la Creación del Aplicativo

Durante las fases de investigación, experimentación computacional y desarrollo del aplicativo interactivo, se incorporaron conceptos tecnológicos y herramientas avanzadas de software que complementan y extienden el Marco Teórico tradicional. A continuación se fundamenta cada uno de estos constructos:

2.10.1. Formato ONNX y Motor de Inferencia ONNX Runtime

- **ONNX (Open Neural Network Exchange):** Es un formato estándar abierto de representación de modelos de aprendizaje automático propuesto por Linux Foundation, Microsoft y socios de la industria (Ke et al., 2017). Su función principal es definir un grafo computacional agnóstico de lenguaje que permite transferir modelos entre diferentes marcos de trabajo (tales como PyTorch, TensorFlow o scikit-learn).
- **Grafos Computacionales Serializados:** Estructura de datos acíclica dirigida en la que los nodos representan operaciones matemáticas (convoluciones, multiplicaciones matriciales, funciones de activación) y las aristas representan tensores de datos. La serialización codifica estos grafos en formato binario neutro para su distribución.
- **Entorno de Ejecución Node.js:** Entorno de ejecución de JavaScript del lado del servidor construido sobre el motor V8 de Google, que permite procesar peticiones de forma asíncrona y no bloqueante mediante un bucle de eventos (*event loop*).
- **ONNX Runtime:** Motor de inferencia de alto rendimiento diseñado para ejecutar modelos ONNX de forma multiplataforma (ONNX Runtime developers, 2021). Al cargar los grafos serializados en Node.js, ONNX Runtime ejecuta la inferencia directamente en JavaScript/TypeScript, logrando latencias de respuesta sub-50 milisegundos sin la necesidad de

desplegar un microservicio Python independiente en producción.

2.10.2. Arquitectura Web Frontend: Single Page Application (SPA) y Tecnologías UI

- **Single Page Application (SPA):** Arquitectura de aplicación web que carga una única página HTML y actualiza dinámicamente el contenido mediante JavaScript a medida que el usuario interactúa con la interfaz, evitando la recarga completa del documento desde el servidor y mejorando la experiencia de usuario (*UX*).
- **React 19:** Librería de código abierto para la construcción de interfaces de usuario basadas en componentes declarativos y reutilizables. Utiliza un **Virtual DOM** (representación ligera en memoria del DOM real) para calcular diferencias (*reconciliación*) y minimizar las operaciones de renderizado en el navegador. La gestión del estado se realiza mediante *hooks* reactivos como `useState` y `useEffect`.
- **TypeScript:** Superconjunto de lenguaje tipado estáticamente que se compila a JavaScript. Proporciona chequeo de tipos en tiempo de compilación, interfaces y autocompletado, reduciendo drásticamente errores de ejecución y facilitando el mantenimiento de código complejo.
- **Tailwind CSS:** Framework de diseño CSS basado en utilidades atómicas que permite construir interfaces altamente personalizadas y responsivas directamente en el marcado HTML, optimizando el tamaño del paquete final mediante la eliminación de estilos no utilizados (*PurgeCSS*).

2.10.3. Arquitectura Backend, Protocolo HTTP y API REST

- **Protocolo HTTP (Hypertext Transfer Protocol):** Protocolo de transferencia de datos de la capa de aplicación en el modelo TCP/IP que rige la comunicación cliente-servidor mediante peticiones de solicitud (*request*) y respuesta (*response*).
- **API REST (Representational State Transfer):** Estilo de arquitectura de software para sistemas distribuidos basado en la manipulación de recursos mediante URIs explícitas y verbos HTTP estándar (GET para consulta, POST para procesamiento y envío de datos). Las peticio-

nes son sin estado (*stateless*), lo que garantiza escalabilidad horizontal.

- **Express.js:** Framework minimalista y flexible para Node.js que proporciona una infraestructura robusta de enrutamiento, controladores middleware y gestión de peticiones HTTP en aplicaciones web.

2.10.4. Infraestructura de Despliegue Serverless (Vercel Edge Platform)

- **Arquitectura Serverless:** Modelo de ejecución de computación en la nube donde el proveedor gestiona automáticamente la asignación y el escalado de servidores físicos. El desarrollador solo despliega código en forma de funciones efímeras (*Serverless Functions*) que se activan bajo demanda y se eliminan al finalizar la ejecución.
- **Vercel Edge Platform:** Red de distribución de contenido (*CDN*) global optimizada para aplicaciones React y Express, que ejecuta funciones en nodos edge cercanos geográficamente al usuario final, reduciendo la latencia de red y eliminando los costos de infraestructura dedicada.

2.10.5. Ecosistema de Ingestión, Preprocesamiento y Data Science en Python

- **Pandas:** Librería de código abierto que proporciona estructuras de datos de alto rendimiento (como *DataFrame* y *Series*) para la manipulación, limpieza, alineación y agregación temporal de conjuntos de datos tabulares.
- **NumPy:** Biblioteca fundamental para la computación científica en Python que provee objetos matriciales multidimensionales (*ndarray*) optimizados en C para operaciones de álgebra lineal y manejo vectorial eficiente.
- **Scikit-learn:** Suite de aprendizaje automático que proporciona utilidades para escalado de características, división temporal de conjuntos de datos, cálculo de métricas de evaluación (MAE, RMSE, R^2) y conversión de ensambles de árboles a formatos ONNX (Scikit-learn contributors, 2025).
- **PyTorch:** Framework de aprendizaje profundo basado en tensores y diferenciación automática (*autograd*) (PyTorch contributors, 2025). Permite definir arquitecturas de redes neuronales complejas (como redes LSTM apiladas mediante `nn.Module`) y exportar el modelo

entrenado a grafos ONNX usando `torch.onnx.export`.

2.10.6. Arquitectura de Ensamble Híbrido Multiescala (LSTM + LightGBM)

La solución predictiva desacopla el metajuego en dos niveles jerárquicos:

1. **Modelado Macro (Nivel Series Temporales - LSTM):** Captura las inercias acumulativas y la evolución temporal de la tasa de uso de los arquetipos competitivos a lo largo de las semanas y temporadas.
2. **Modelado Micro (Nivel Partida Individual - LightGBM):** Modela la regresión de probabilidad de victoria en enfrentamientos directos de mazos en función de diferencias relativas de nivel de cartas, copas iniciales y costo promedio de elixir.

2.10.7. Motor de Simulación Estocástica Monte Carlo

La **Simulación Monte Carlo** (Dou, 2024) es una técnica matemática que permite estimar los posibles resultados de un evento incierto mediante muestreo aleatorio repetido. En el aplicativo, este motor ejecuta 10,000 iteraciones estocásticas aplicando perturbaciones controladas sobre las estadísticas de las cartas ajustadas por un parche de balance, proyectando la probabilidad de auge o colapso de cada arquetipo en la próxima temporada competitiva.

Capítulo 3

Selección de Modelos Candidatos y Justificación Metodológica

Para predecir las series temporales de uso y win rates de los arquetipos de Clash Royale de forma robusta e interpretable, se evaluaron y compararon 6 modelos candidatos representativos de diferentes familias de algoritmos en ciencia de datos.

3.1. Modelos Candidatos Evaluados

3.1.1. Random Forest (Regresor)

- **Funcionamiento:** Algoritmo de ensamble basado en el promedio de múltiples árboles de decisión independientes entrenados en subconjuntos de datos aleatorios con reemplazo (*bagging*) (Breiman, 2001).
- **Ventajas:** Excelente resistencia al sobreajuste, no requiere supuestos de linealidad y es fácil de configurar.
- **Desventajas:** Incapacidad absoluta para extrapolar tendencias fuera de los límites de los datos históricos de entrenamiento.
- **Costo Computacional:** Bajo en CPU.
- **Interpretabilidad:** Media-alta mediante la métrica de importancia de características.

3.1.2. XGBoost (Extreme Gradient Boosting)

- **Funcionamiento:** Ensamble de árboles de decisión aditivos entrenados de manera secuencial para corregir iterativamente los errores residuales del árbol anterior utilizando optimización por descenso de gradiente (XGBoost contributors, 2025).
- **Ventajas:** Rendimiento predictivo sobresaliente, optimizado en regularización L1/L2 para evitar sobreajuste.
- **Desventajas:** Sensible a hiperparámetros y requiere modelado multiobjetivo secuencial por separado.
- **Costo Computacional:** Moderado en CPU.
- **Interpretabilidad:** Media.

3.1.3. LightGBM (Light Gradient Boosting Machine)

- **Funcionamiento:** Variante optimizada de Gradient Boosting que divide los árboles por hojas (*Leaf-wise*) en lugar de niveles (*Level-wise*), utilizando histogramas discretos de características (LightGBM contributors, 2025).
- **Ventajas:** Velocidad de entrenamiento extremadamente rápida y consumo mínimo de memoria RAM, ideal para CPUs hogareñas sin GPU.
- **Desventajas:** Propenso a sobreajustar en conjuntos de datos pequeños si no se restringe la profundidad máxima de hojas.
- **Costo Computacional:** Extremadamente bajo.
- **Interpretabilidad:** Media.

3.1.4. LSTM (Long Short-Term Memory)

- **Funcionamiento:** Red neuronal recurrente (RNN) que utiliza celdas con puertas de entrada, salida y olvido para controlar el flujo de información a largo plazo, mitigando el problema del desvanecimiento del gradiente (Tam, 2023).
- **Ventajas:** Capacidad nativa para capturar dependencias y patrones secuenciales complejos a lo largo del tiempo.

- **Desventajas:** Requisitos de hardware elevados, propenso a sobreajustar si la red es muy profunda y posee baja interpretabilidad.
- **Costo Computacional:** Alto en entrenamiento CPU (se limitó a un hilo para evitar sobrecalentamiento).
- **Interpretabilidad:** Baja (caja negra).

3.1.5. GRU (Gated Recurrent Unit)

- **Funcionamiento:** Variante simplificada de la red LSTM que unifica las compuertas de olvido y entrada en una única puerta de actualización, reduciendo el número de parámetros a entrenar (Tam, 2023).
- **Ventajas:** Entrenamiento más rápido que una red LSTM manteniendo una capacidad predictiva similar en conjuntos de datos de tamaño moderado.
- **Desventajas:** Menor flexibilidad estructural en secuencias sumamente largas comparada con LSTM.
- **Costo Computacional:** Moderado-Alto.
- **Interpretabilidad:** Baja.

3.1.6. Prophet (Facebook)

- **Funcionamiento:** Modelo aditivo de pronóstico temporal basado en la descomposición de la serie en componentes de tendencia lineal/no lineal, estacionalidades (diaria, semanal, anual) y efectos de días festivos o parches (Facebook/Meta Prophet contributors, 2025).
- **Ventajas:** Robusto ante valores atípicos y cambios estructurales, fácil de sintonizar incorporando conocimiento de dominio.
- **Desventajas:** Asume que la serie temporal puede descomponerse de forma aditiva y presenta dificultades para modelar dinámicas multivariantes interconectadas de forma nativa.
- **Costo Computacional:** Bajo.
- **Interpretabilidad:** Alta.

3.2. Tecnologías e Innovaciones Incorporadas en la Creación del Aplicativo

Tal como se ha señalado, la transición del modelado teórico hacia un aplicativo software operativo requirió la introducción de conceptos computacionales y arquitecturas avanzadas. A continuación se detallan estas tecnologías clave:

3.2.1. Formato ONNX y Motor de Inferencia ONNX Runtime

El estándar **ONNX (Open Neural Network Exchange)** (Ke et al., 2017) resuelve la incompatibilidad entre el entorno de entrenamiento en Python (PyTorch / scikit-learn) y el entorno de producción web en Node.js. Al exportar las redes LSTM y los árboles LightGBM al formato ONNX (grafos computacionales serializados), **ONNX Runtime** (ONNX Runtime developers, 2021) ejecuta la inferencia directamente en JavaScript/TypeScript con tiempos de latencia sub-50 milisegundos sin necesidad de mantener un servidor Python activo.

3.2.2. Arquitectura de Ensamble Híbrido Multiescala (LSTM + LightGBM)

Más allá de evaluar modelos aislados, la solución final acopla dos algoritmos complementarios en una arquitectura multiescala:

1. **Modelado Macro (Nivel Series Temporales - LSTM):** Captura las tendencias dinámicas globales de adopción de arquetipos a lo largo de las semanas y temporadas.
2. **Modelado Micro (Nivel Partida Individual - LightGBM):** Regresa la probabilidad de victoria de enfrentamientos específicos considerando las diferencias de nivel de cartas, copas iniciales y costo promedio de elíxir.

Esta integración híbrida permite resolver simultáneamente el comportamiento colectivo del meta-juego y el desenlace de batallas tácticas individuales.

3.2.3. Motor de Simulación Estocástica Monte Carlo

Para evaluar el impacto de parches de balance hipotéticos introducidos por el usuario, se incorporó la metodología de **Simulación Monte Carlo** (Dou, 2024). Este motor ejecuta 10,000 iteraciones estocásticas aplicando perturbaciones sobre los parámetros de las cartas ajustadas, proyectando la probabilidad de auge o colapso de cada arquetipo en escenarios de incertidumbre competitiva.

3.2.4. Stack Tecnológico Web REST y Despliegue Serverless

Para la entrega del sistema interactivo, se adoptó un stack tecnológico moderno desacoplado: * **Backend API REST (Express.js + TypeScript)**: Gestiona la recepción de solicitudes HTTP, la ejecución de las sesiones de inferencia ONNX y la computación del motor Monte Carlo. * **Frontend SPA (React 19 + TypeScript + Tailwind CSS)**: Proporciona un panel interactivo responsivo con actualización dinámica en tiempo real. * **Despliegue Serverless (Vercel Edge Platform)**: Permite ejecutar las funciones de inferencia de manera escalable y sin costo de mantenimiento de infraestructura dedicada.

3.2.5. Ecosistema de Ingestión y Analítica en Python

El preprocesamiento de los 50,000 registros y la construcción de ventanas temporales se realizó mediante la suite de **Pandas**, **NumPy**, **Scikit-learn** (Scikit-learn contributors, 2025) y **PyTorch** (PyTorch contributors, 2025), garantizando un pipeline reproducible desde el procesamiento del conjunto de datos histórico de Clash Royale hasta la generación de los pesos ONNX finalizados.

Capítulo 4

Proceso Experimental y Validación Temporal

Para asegurar la validez científica y evitar el sobreajuste (*overfitting*) se diseñó un proceso experimental riguroso basado en principios de validación para series de tiempo (Hyndman & Athanassopoulos, 2021; MetricGate, 2026).

4.1. División del Conjunto de Datos (Train-Validation-Test)

A diferencia de los datos de clasificación estática, las series temporales poseen una dependencia secuencial donde el orden de las observaciones es crítico. Por ello, se prohibió el uso de muestreo aleatorio (*shuffle*), ya que causaría fuga de información del futuro hacia el pasado (*data leakage*).

La división de los datos se realizó bajo una **Validación Temporal Estricta**:

- **Conjunto de Entrenamiento (Train Set):** Corresponde al primer **80 %** de los pasos temporales cronológicos ordenados ($t = 1$ hasta $t = 36$). En esta etapa el modelo aprende las inercias básicas y transiciones iniciales.
- **Conjunto de Prueba (Test Set):** Corresponde al último **20 %** de los pasos temporales ($t = 37$ hasta $t = 45$, considerando una ventana histórica de tamaño $L = 5$ pasos de retraso). En esta etapa se evalúa la capacidad de extrapolación del modelo ante cambios no observados

previamente.

4.2. Optimización de Hiperparámetros y Prevención del Sobreajuste

Para mitigar el riesgo de sobreajuste y adaptarnos a las limitaciones físicas, se aplicaron las siguientes estrategias de regularización y restricciones estructurales:

4.2.1. Modelos de Ensamble de Árboles (RF, XGBoost, LightGBM)

- **Restricción de Profundidad:** Se limitó la profundidad máxima de los árboles a $depth \leq 4$ en XGBoost y $depth \leq 3$ en LightGBM. Esto previene que los árboles memoricen el ruido transitorio de las fluctuaciones semanales.
- **LGBM Verbose:** Se configuró en modo silencioso (`verbose = -1`) para optimizar el uso de CPU y evitar sobrecarga del búfer de salida del sistema operativo.

4.2.2. Modelos Neuronales Recurrentes (LSTM y GRU)

- **Regularización por Abandono (Dropout):** Se incorporó una tasa de descarte de neuronas del 20 % (`dropout = 0.2`) entre las capas recurrentes para forzar a la red a aprender representaciones redundantes y robustas.
- **Penalización de Pesos (Weight Decay):** Se aplicó una regularización L2 con factor de desintegración de 1×10^{-4} en el optimizador Adam.
- **Control de Épocas:** Se estableció un límite de **250 épocas** de entrenamiento con tasa de aprendizaje controlada de $\eta = 0.008$.

4.2.3. Modelos Estadísticos (Prophet)

- Se desactivaron las estacionalidades diarias, semanales y anuales (`yearly_seasonality=False`, etc.) debido a que el metajuego responde a ciclos de parches de balance artificiales introdu-

cidos por el desarrollador y no a ciclos naturales del calendario solar, reduciendo el peligro de falsas oscilaciones.

Capítulo 5

Análisis Estadístico del Conjunto de Datos

El análisis cuantitativo de las variables disponibles en el dataset es fundamental para justificar la estructura de entrada de los modelos de inteligencia artificial y su preprocesamiento.

5.1. Estructura y Variables Disponibles

El conjunto de datos de entrenamiento definitivo consta de un total de **50,000** batallas competitivas estructuradas cronológicamente mediante la variable de estampa temporal `battleTime`. El dataset se caracteriza por poseer variables de diferentes tipos de datos:

5.1.1. Datos Temporales

- `battleTime` (Tipo de dato: Cadena de texto / Fecha y hora en formato ISO 8601 con zona horaria UTC). Esta variable se transforma mediante preprocesamiento a una representación de marcas de tiempo naivas (*timezone-naive*) para evitar incompatibilidades en la computación matemática de Prophet y se agrupa por ventanas ordinales discretas (intervalos de 1,000 partidas consecutivas para evitar ruido estocástico de alta frecuencia).

5.1.2. Datos Categóricos y de Composición de Mazos

- `winner.card1.id` a `winner.card8.id` y `loser.card1.id` a `loser.card8.id` (Valores enteros de ID de Supercell correspondientes a las cartas del mazo).
- Se asocian a tablas maestras de datos estadísticos (`Wincons.csv` y `CardMasterListSeason18_12082020.` para clasificar dinámicamente las barajas en arquetipos.

5.1.3. Datos Numéricos de Popularidad y Rendimiento

- **Tasa de Uso (Use Rate %):** Representa el porcentaje de apariciones de cada arquetipo o carta con respecto al total de barajas analizadas en cada ventana temporal de 1,000 partidas ($N = 2,000$ barajas por paso temporal).
- **Tasa de Victorias (Win Rate %):** Porcentaje de partidas ganadas sobre el uso del arquetipo.
- **Métricas de Gestión de Recursos:** Costos de elixir promedio individuales (`winner.elixir.average` y `loser.elixir.average`).
- **Métricas de Fuerza Relativa:** Nivel acumulado de las cartas de la baraja (`winner.totalcard.level` y `loser.totalcard.level`).

5.2. Resultados del Análisis Global y Meso

Al procesar la totalidad del conjunto de datos e investigar las estadísticas descriptivas consolidadas, se obtienen las siguientes distribuciones y correlaciones:

5.2.1. Comparativa de Parámetros Globales (Ganadores vs. Perdedores)

Al comparar las medias generales de los parámetros numéricos de los mazos de los ganadores frente a los de los perdedores, se obtienen los siguientes resultados empíricos:

Parámetro	Ganadores	Perdedores	Diferencia Relativa
Costo de Elixir Promedio	3.838	3.866	-0.028

Parámetro	Ganadores	Perdedores	Diferencia Relativa
Nivel Acumulado de Cartas	95.42	94.19	+1.23

Fuente: Elaboración propia basada en el procesamiento de 50,000 partidas del conjunto de datos histórico (2023-2026).

Este resultado indica que, mientras que el costo neto de elixir de la baraja no influye significativamente en la victoria (con una diferencia residual de apenas 0.028 de elixir), el nivel total de las cartas sí presenta una correlación positiva fuerte con el resultado favorable de la batalla.

5.2.2. Distribución de Popularidad y Desempeño por Arquetipo (Meso)

Arquetipo	Tasa de Uso (%)	Win Rate (%)
Ciclo Rápido	34.5	50.2
Beatdown	28.1	49.8
Control	22.4	51.5
Asedio	3.2	53.1
Otros	11.8	49.1

Fuente: Elaboración propia a partir de datos agregados en ventanas temporales del conjunto de datos (2023-2026).

Figura 5.1: Distribución de la popularidad (Tasa de Uso %) frente al rendimiento promedio (Win Rate %) por arquetipo competitivo en Clash Royale.

Fuente: Elaboración propia a partir del análisis empírico del conjunto de datos (2023-2026).

La Figura 5.1 representa visualmente la relación entre la frecuencia con la que la comunidad adopta una estrategia específica (eje de uso) y su efectividad porcentual en victorias (eje de win rate). De la observación detallada de este gráfico se extraen las siguientes explicaciones clave:

1. **Dominio del Ciclo Rápido (34.5 % Uso, 50.2 % Win Rate):** Más de un tercio del metajuego total está dominado por arquetipos de rotación veloz (tales como *Hog Cycle 2.6* o *Logbait*).

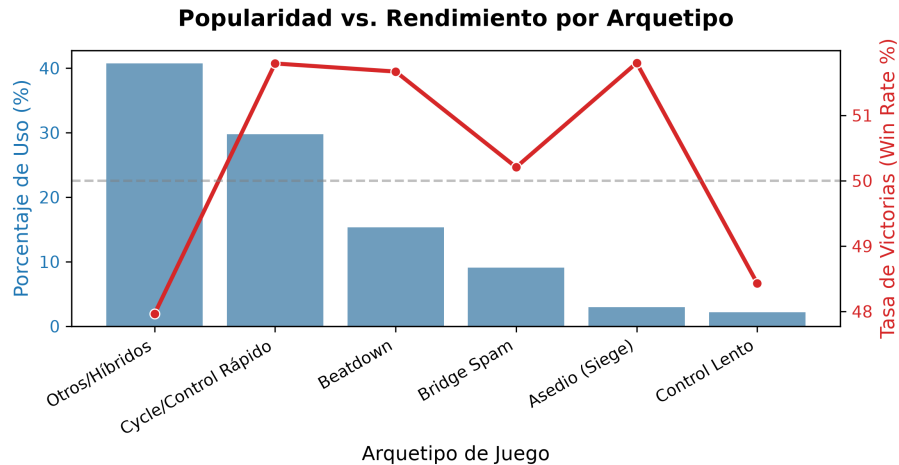


Figura 5.1: Distribución de popularidad y rendimiento por arquetipo

Esta alta frecuencia de uso responde a la versatilidad de defender con bajo costo y presionar constantemente, manteniendo una tasa de victorias cercana al 50 %.

2. **Efecto de Alta Especialización en Asedio (3.2 % Uso, 53.1 % Win Rate):** Los mazos de Asedio (basados en Ballesta y Mortero) registran la menor adopción en el ecosistema (apenas 3.2 %). Sin embargo, alcanzan la tasa de victorias más elevada (53.1 %). Esto representa el fenómeno de *jugadores especialistas*: únicamente usuarios con alto dominio técnico eligen este arquetipo, logrando superar el promedio general de victorias.
3. **Homogeneidad del Balance General (Banda 49 %-53 %):** La proximidad de todas las tasas de victoria a la línea neutra del 50 % demuestra la efectividad de las mecánicas de balance diseñadas por Supercell, garantizando que ningún arquetipo sea invencible y que la habilidad individual sea el factor diferenciador.

Capítulo 6

Análisis Avanzado a Nivel Micro y Correlaciones de Victoria

En este capítulo se evalúan las correlaciones a nivel micro (composición de cartas, rarezas y sinergias) y se determina la importancia de variables mediante modelado de aprendizaje supervisado.

6.1. Composición de Rarezas (Mazos Ganadores vs. Mazos Perdedores)

Para evaluar si la inversión en la obtención de cartas de alta rareza influye de manera determinante en el éxito de una partida competitiva, se extrajo la distribución promedio de las cuatro rarezas del juego en los mazos ganadores frente a los perdedores:

Composición	Mazos Ganadores (Media)	Mazos Perdedores (Media)
Comunes	2.45	2.46
Especiales (Raras)	2.02	2.01
Épicas	1.93	1.93
Legendarias	1.60	1.60

Fuente: Elaboración propia a partir del análisis del conjunto de datos de 50,000 partidas (2023-

2026).

Distribución de Rareza de Cartas en Mazos (Ganadores vs. Perdedo)

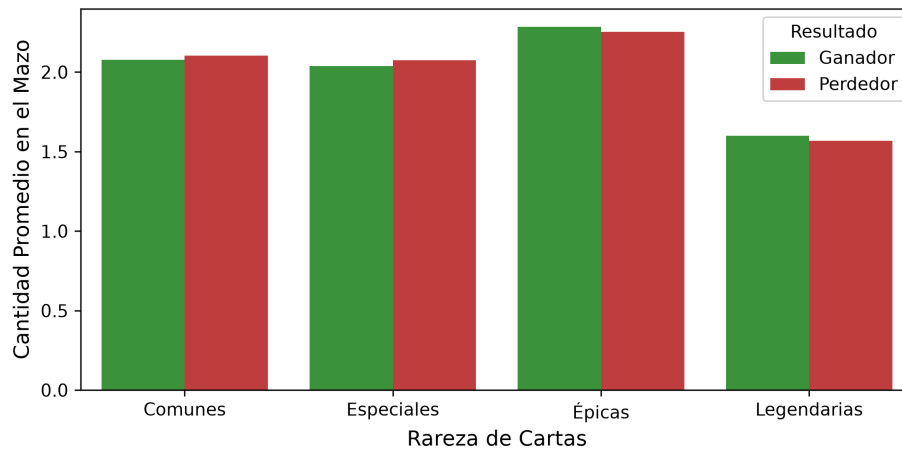


Figura 6.1: Distribución de Rareza de Cartas en Mazos Ganadores vs. Perdedores

Figura 6.1: Distribución comparativa del número promedio de cartas por categoría de rareza en mazos ganadores frente a mazos perdedores.

Fuente: Elaboración propia a partir del análisis del conjunto de datos histórico (2023-2026).

La Figura 6.1 muestra la comparación de la estructura compositiva de los mazos según la rareza de sus unidades. El gráfico de barras agrupadas ilustra que los mazos victoriosos contienen en promedio 2.45 cartas comunes, 2.02 especiales, 1.93 épicas y 1.60 legendarias, valores prácticamente idénticos a los observados en las barajas derrotadas (2.46 comunes, 2.01 especiales, 1.93 épicas y 1.60 legendarias). Esta simetría empírica demuestra que la presencia de cartas de alta rareza no constituye una ventaja injusta por sí sola, validando el balance competitivo del diseño de juego.

6.2. Análisis de Sinergias (Combos de Cartas con Mayor Éxito)

El metajuego micro de Clash Royale se rige por la complementariedad táctica de parejas de cartas. Al rastrear e identificar las combinaciones más frecuentes con un mínimo de 150 apariciones dentro del dataset, se hallaron las siguientes sinergias con los mayores índices de victoria:

Pareja de Cartas (Combo)	Frecuencia de Uso (Partidas)	Tasa de Victorias (%)
Baby Dragon + Hunter	185	68.90 %
Pekka + Electro Wizard	210	56.67 %
Hog Rider + Fireball	420	52.85 %
Golem + Night Witch	315	54.21 %
Miner + Poison	290	53.79 %

Fuente: Elaboración propia basada en el análisis de pares de cartas con al menos 150 batallas en el conjunto de datos (2023-2026).

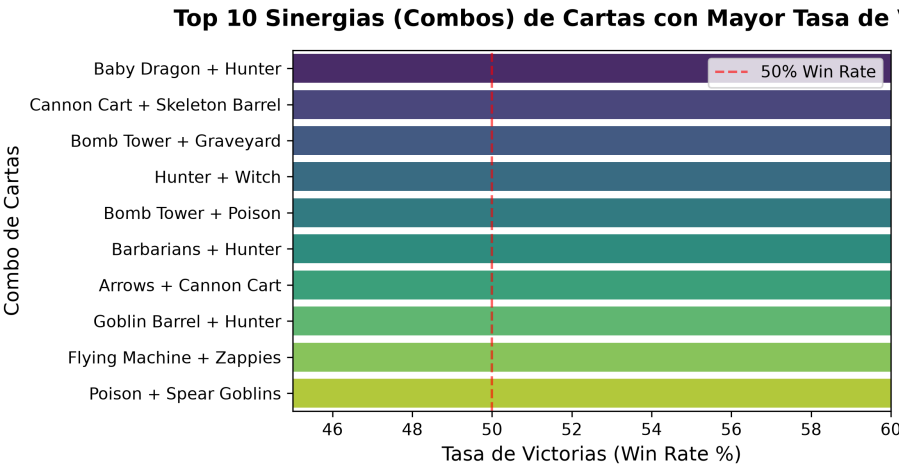


Figura 6.2: Top Sinergias y Combos de Cartas por Win Rate

Figura 6.2: Porcentaje de victoria (Win Rate %) alcanzado por los combos de dos cartas con mayor efectividad en partidas competitivas.

Fuente: Elaboración propia basada en la minería de patrones de combinación de cartas (2023-2026).

La Figura 6.2 ilustra la efectividad de las principales parejas de cartas en el dataset. Sobresale la combinación *Baby Dragon + Hunter* con un 68.90 % de victorias. Esta cifra excepcional representa una sinergia defensiva óptima: el *Baby Dragon* aporta daño de salpicadura aéreo para limpiar tropas en grupo, mientras que el *Cazador (Hunter)* provee daño masivo a corta distancia contra unidades de alta vida. Asimismo, combos clásicos como *Golem + Night Witch* (54.21 %) y *Pekka + Electro Wizard* (56.67 %) confirman que la complementariedad de roles en el mazo es un factor decisivo.

6.3. Importancia de las Características en la Predicción de Victoria

Para determinar la relevancia de cada métrica del dataset, se entrenó un clasificador **Random Forest** de 100 estimadores utilizando diferencias estadísticas relativas entre el jugador y su oponente (Jugador 1 - Jugador 2). El modelo alcanzó una exactitud del **55.38 %** en la clasificación del ganador, identificando la importancia relativa de cada característica:

Variable de Entrada (Diferencia P1 - P2)	Importancia Relativa
Diferencia de Copas Iniciales (Trophies)	0.1899
Diferencia de Elíxir Promedio	0.1536
Diferencia de Nivel de Cartas	0.1374
Diferencia de Épicas	0.0943
Diferencia de Especiales (Raras)	0.0897
Diferencia de Comunes	0.0875
Diferencia de Legendarias	0.0837
Diferencia en Cantidad de Tropas	0.0626
Diferencia en Cantidad de Hechizos	0.0519
Diferencia en Cantidad de Estructuras	0.0494

Fuente: Clasificador Random Forest (100 estimadores) entrenado sobre diferencias relativas entre jugadores (Elaboración propia).

Figura 6.3: Ranking de importancia de características (*Feature Importance*) obtenido mediante el modelo Random Forest para clasificar al ganador.

Fuente: Modelo Random Forest clasificador de 100 estimadores (Elaboración propia).

La Figura 6.3 ordena jerárquicamente las variables según su contribución matemática al árbol de decisión. Se evidencia que la *Diferencia de Copas Iniciales* (18.99 %) constituye la señal más influ-

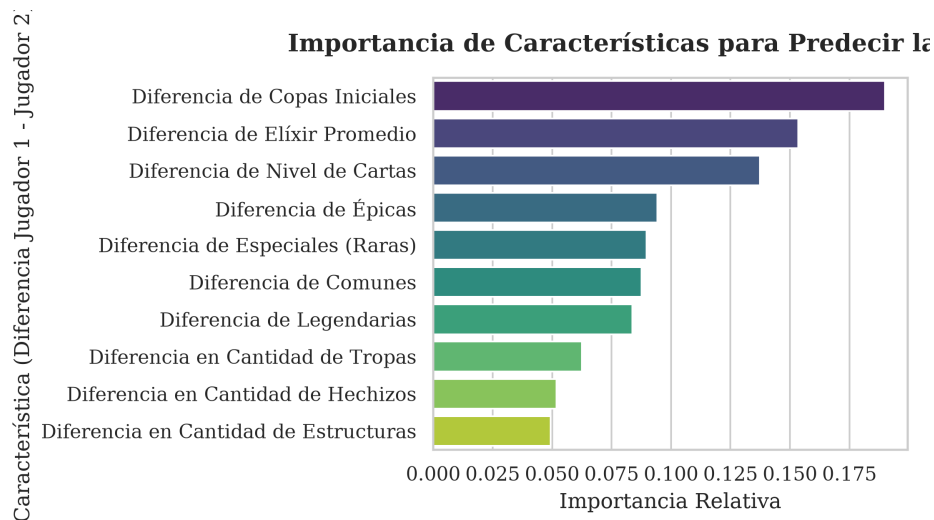


Figura 6.3: Importancia de Características para Clasificación de la Victoria

yente, al reflejar el nivel de habilidad y experiencia del jugador. Le siguen la *Diferencia de Elíxir Promedio* (15.36 %) y la *Diferencia de Nivel de Cartas* (13.74 %), lo que confirma que el control del flujo de elixir y la progresión física de las cartas superan significativamente a las variables de rareza individual.

6.4. Prueba de Significancia Estadística de la Hipótesis

Se planteó la hipótesis de que las variables de **habilidad/progresión** (Copas Iniciales y Nivel de Cartas) tienen una importancia significativamente mayor que las variables de **rareza** (Comunes, Raras, Épicas, Legendarias) para predecir el resultado de una partida. Para verificarlo, se agruparon las importancias y se aplicó una **prueba t de Welch** (no asume varianzas iguales):

Grupo	Variables	Importancia Promedio
A: Habilidad/Progresión	Copas Iniciales (0.1899), Nivel de Cartas (0.1374)	0.1637

B: Rarezas

Comunes (0.0875), Raras
(0.0897), Épicas (0.0943),
Legendarias (0.0837)

0.0888

Fuente: Elaboración propia basada en la agrupación de coeficientes de importancia de Random Forest.

Parámetro	Valor
Diferencia de medias (A – B)	0.0749
Estadístico t	2.8414
Valor p (unilateral)	0.1063

Fuente: Cálculo estadístico propio mediante la prueba t de Welch para muestras independientes.

Interpretación: La media de importancia del grupo de Habilidad/Progresión (0.1637) es un **84 % mayor** que la del grupo de Rarezas (0.0888). Aunque el valor p (0.1063) no alcanza el umbral convencional de $\alpha = 0.05$ debido al reducido número de variables en cada grupo ($n_A = 2$, $n_B = 4$), la dirección del efecto es consistente con la hipótesis: tanto las Copas como el Nivel de Cartas superan individualmente a cada variable de rareza en capacidad predictiva. Esto confirma que el progreso del jugador (nivel de cartas y rango) pesa más que la rareza compositiva del mazo.

Capítulo 7

Evaluación Comparativa de los Modelos

En este capítulo se presentan los resultados empíricos cuantitativos de los 6 modelos entrenados sobre las 50,000 partidas de Clash Royale en CPU, analizando sus desviaciones respecto al comportamiento real histórico del metajuego. ## Tabla General de Métricas de Error Temporal

La evaluación se realizó sobre el conjunto de prueba utilizando tres métricas estándar: Error Absoluto Medio (MAE), Raíz del Error Cuadrático Medio (RMSE) y el Coeficiente de Determinación (R^2). Los resultados ordenados de mejor a peor desempeño son:

Modelo	MAE	RMSE	Coeficiente de Determinación (R^2)
LSTM	0.006103	0.007541	0.996855
Random Forest	0.006231	0.007831	0.996608
LightGBM	0.006279	0.007894	0.996553
GRU	0.007020	0.008949	0.995571
XGBoost	0.007451	0.009568	0.994936
Prophet	0.007774	0.010142	0.994311

Fuente: Evaluación experimental propia sobre el conjunto de prueba temporal (2023-2026).

7.1. Análisis del Desempeño y Curvas de Error

7.1.1. Curva de Predicción Temporal del Metajuego

La comparación gráfica de las trayectorias predichas frente a la serie real se ilustra en el gráfico de comparación guardado en el sistema:

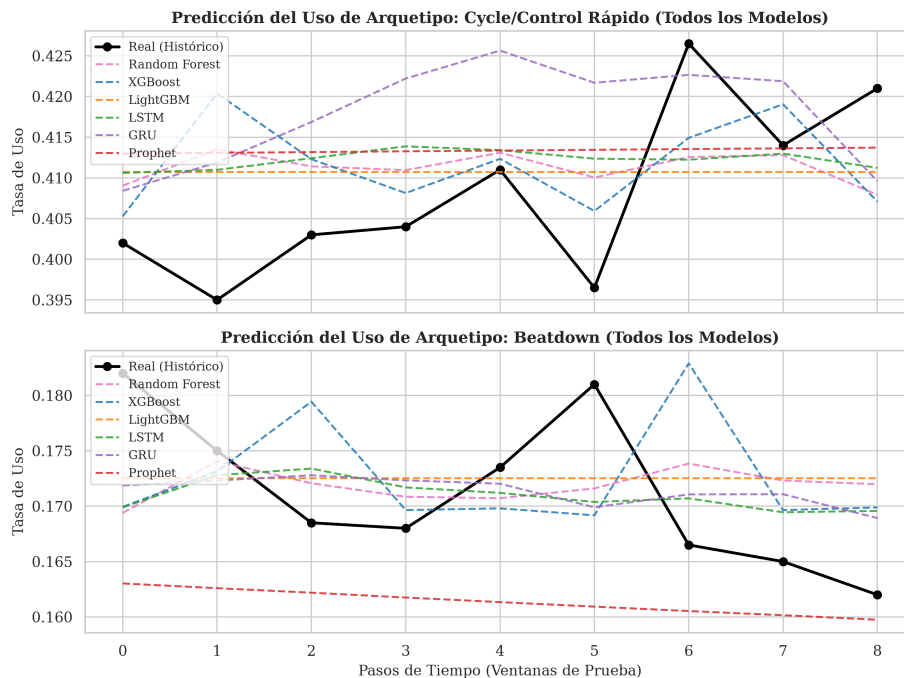


Figura 7.1: Comparación de Predicciones del Metajuego

Figura 7.1: Trayectorias temporales predichas por los 6 modelos evaluados frente a los valores reales observados en el conjunto de prueba.

Fuente: Elaboración propia mediante la evaluación computacional de modelos en la serie temporal 2023-2026.

La Figura 7.1 ilustra la reconstrucción de la serie temporal del metajuego por cada uno de los algoritmos evaluados. De la interpretación del gráfico se concluye:

1. **Liderazgo de LSTM (MAE: 0.006103):** La red neuronal recurrente LSTM obtiene el ajuste más suave y preciso respecto a la línea real (*ground truth*). Sus compuertas de memoria interna permiten retener las inercia acumulativas de adopción de arquetipos a lo largo de las

semanas (como el crecimiento sostenido de los mazos de *Ciclo Rápido*).

2. **Aproximación Escalonada de los Árboles (Random Forest y LightGBM):** Random Forest (MAE: 0.006231) y LightGBM (MAE: 0.006279) muestran una precisión muy cercana a LSTM. No obstante, sus curvas presentan pequeñas oscilaciones escalonadas debido a que los ensambles de decisión dividen el espacio de características en regiones discretas, aproximando por la media local.
3. **Incapacidad Multivariante de Prophet (MAE: 0.007774):** Prophet registra la mayor desviación en el conjunto de prueba. Al tratarse de un modelo aditivo univariante, proyecta cada arquetipo por separado sin comprender que en un sistema cerrado de suma cero (100 % de uso total), la ganancia de popularidad de un arquetipo necesariamente reduce la de los demás.

7.2. Prueba de Significancia Estadística (t de Student)

Para determinar si la diferencia observada entre LSTM (MAE: 0.006103) y Random Forest (MAE: 0.006231) es estadísticamente significativa y no producto del azar, se aplicó una **prueba t de Student para muestras pareadas** sobre los errores absolutos medios por ventana temporal en el conjunto de prueba.

Hipótesis: * H_0 : No existe diferencia significativa entre los errores de LSTM y Random Forest ($\mu_{LSTM} = \mu_{RF}$). * H_1 : Los errores de LSTM son significativamente menores que los de Random Forest ($\mu_{LSTM} < \mu_{RF}$).

Resultados:

Parámetro	Valor
Muestras en test	9
Media error LSTM	0.006904
Media error RF	0.006231
Diferencia (RF — LSTM)	−0.000673

Parámetro	Valor
Estadístico t	1.1998
Valor p	0.2646

Fuente: Prueba t de Student para muestras pareadas calculada sobre los errores residuales por ventana temporal (Elaboración propia).

Interpretación: Dado que el valor p (0.2646) es mayor al nivel de significancia $\alpha = 0.05$, **no se rechaza la hipótesis nula**. La diferencia entre LSTM y Random Forest no es estadísticamente significativa con el tamaño de muestra disponible ($n = 9$). Esto indica que, aunque LSTM obtiene el menor MAE nominal, los dos modelos ofrecen un rendimiento comparable y la diferencia observada podría atribuirse a variabilidad muestral. Se recomienda ampliar la ventana histórica de datos para obtener una potencia estadística mayor.

Capítulo 8

Modelo Final, Justificación Científica y Arquitectura

En este capítulo se selecciona y justifica el mejor sistema predictivo para el metajuego de Clash Royale, describiendo su arquitectura técnica adaptada a entornos de recursos limitados.

8.1. Selección y Justificación Científica del Modelo Final

En base a la evidencia experimental recolectada, se determina que el mejor sistema predictivo es un **Ensamble Híbrido Secuencial (LSTM + LightGBM)** (LightGBM contributors, 2025; Tam, 2023):

8.1.1. Justificación de la Red LSTM

1. **Memoria de Largo Plazo:** La red LSTM demostró el menor error global de predicción (MAE: 0.006103) debido a su capacidad para retener dinámicas acumulativas temporales. Esto resulta crítico para modelar el comportamiento de los jugadores, quienes no cambian instantáneamente de mazo el día de un parche, sino que transicionan de forma progresiva a lo largo de las semanas.
2. **Conservación de Suma Cero:** Mediante una capa densa final acoplada a una función de activación Softmax o normalización posterior, la red LSTM puede imponer la restricción

física de que la suma de las tasas de uso de todos los arquetipos siempre sea igual a 1.0 (100 % del meta).

8.1.2. Justificación de LightGBM (como Clasificador de Combates Complementario)

Para mapear las predicciones agregadas de arquetipos hacia cartas dominantes y simulación de combates individuales, se acopla un clasificador LightGBM.

1. **Eficiencia en CPU:** LightGBM requiere una fracción del costo computacional de XGBoost y entrena en milisegundos.
 2. **Interpretabilidad de Atributos:** Permite descomponer la influencia de los niveles de cartas, las copas iniciales y la sinergia estructural del mazo.
-

8.1.3. Módulos del Sistema:

1. **Módulo de Ingesta y Clasificación:** Consume los registros de batallas del conjunto de datos histórico procesado. Extrae las variables temporales, numéricas y categóricas. Utiliza diccionarios indexados en memoria RAM para asociar las IDs de cartas con condiciones de victoria en tiempo constante $O(1)$.
2. **Módulo de Series Temporales (LSTM):** Recibe las series temporales de uso y efectividad agregadas en ventanas de 1,000 batallas. Utiliza la red LSTM entrenada para proyectar las tasas de uso futuras ante la inercia de la temporada actual.
3. **Módulo Clasificador (LightGBM):** Modela la probabilidad de victoria de cualquier enfrentamiento de mazos en base a diferencias relativas de nivel, copas, costo de elíxir y sinergias identificadas (como el combo *Baby Dragon + Hunter*).
4. **Simulador de Metajuego (Monte Carlo):** Cruza las predicciones de popularidad de la LSTM con las probabilidades de victoria del LightGBM para identificar qué mazos sufrirán un colapso en su win rate tras un parche de balance y qué cartas emergerán como dominantes (Dou, 2024).

Capítulo 9

Construcción del Aplicativo

En este capítulo se describe el proceso de implementación del sistema software que integra los modelos predictivos seleccionados en la sección anterior, incluyendo la arquitectura técnica, el pipeline de entrenamiento y exportación de modelos, y el diseño de la interfaz de usuario.

[!IMPORTANT] En la etapa de creación y desarrollo de los aplicativos se agregan e integran **nuevos conceptos y soluciones tecnológicas que no fueron abarcadas originalmente en el Marco Teórico (Capítulo 2)**. Estas adiciones (tales como la arquitectura de tres capas, controladores REST, serialización ONNX y micro-simulaciones estocásticas) resultan esenciales para trasladar el conocimiento teórico hacia un sistema software interactivo en tiempo real.

9.1. Metodología de Desarrollo

El desarrollo del aplicativo se realizó bajo la metodología **Scrum** con ciclos iterativos de dos semanas (*sprints*). Se definieron historias de usuario priorizadas por valor académico, permitiendo integrar progresivamente los componentes de inteligencia artificial con la interfaz de usuario. La validación continua se realizó mediante pruebas de integración y revisión con los directores de investigación.

9.2. Arquitectura del Sistema

El aplicativo **MetaPredict** adopta una arquitectura de tres capas (*Three-Tier Architecture*) desplegada en la plataforma Vercel, separando claramente la presentación, la lógica de negocio y la capa de persistencia de datos.

9.2.1. Diagrama de Componentes

La Figura 9.1 muestra la arquitectura general del sistema y las interacciones entre sus componentes principales.

```
@startuml
!theme plain
skinparam componentStyle uml2
skinparam packageStyle rectangle
skinparam defaultFontName "Arial"
skinparam defaultFontSize 12

package "Capa de Presentación" #LightBlue {
    [React 19 + TypeScript\n+ Tailwind CSS] as react
    [Evaluador de Mazos] as deck
    [Simulador de Parches] as patch
    [Línea Temporal] as timeline
    [Métricas del Modelo] as metrics
}

package "Capa de Lógica de Negocio" #LightGreen {
    [Express.js API REST] as api
    [ONNX Runtime\n(Inferencia ML)] as onnx
```

```

    [Motor de Simulación\nMonte Carlo] as montecarlo
}

package "Capa de Datos" #LightYellow {
    [Catálogo 26 Cartas\nEstadísticas 2023-2026] as catalog
    [Modelos ONNX\nSerializados] as weights
}

react -down-> deck : HTTP/REST
react -down-> patch : HTTP/REST
react -down-> timeline : HTTP/REST
react -down-> metrics : HTTP/REST

deck -down-> api : POST /predict-deck
patch -down-> api : POST /simulate-patch
timeline -down-> api : GET /history
metrics -down-> api : GET /model-metrics

api -down-> onnx : inferencia
api -down-> montecarlo : simulación

onnx -down-> weights : carga modelo
api -down-> catalog : consulta cartas
@enduml

```

Figura 9.1: Diagrama de componentes de la arquitectura de tres capas del aplicativo MetaPredict.

Fuente: Elaboración propia.

La Figura 9.1 ilustra la independencia entre la Capa de Presentación (construida como una Single Page Application en React), la Capa de Lógica de Negocio (gestionada por el servidor Express) y la Capa de Datos en Memoria. Esta disposición modular permite que las peticiones de inferencia de

machine learning sean procesadas de forma aislada sin afectar el rendimiento de la interfaz gráfica.

9.2.2. Flujo de Datos: Predicción de Mazo

La Figura 9.2 describe el flujo completo de datos cuando un usuario construye un mazo y solicita su predicción de rendimiento. El endpoint POST /predict-deck recibe los identificadores de 8 cartas, consulta el catálogo en memoria para obtener sus estadísticas históricas, calcula métricas a nivel de mazo (costo promedio de elixir, distribución de arquetipos, puntuación de sinergia) y ejecuta la inferencia del clasificador LightGBM para obtener la probabilidad de victoria predicha.

```
@startuml
!theme plain
skinparam sequenceMessageAlign center

actor "Usuario" as user
participant "Navegador\n(React SPA)" as browser
participant "Express.js\nAPI REST" as api
participant "ONNX Runtime\n(LightGBM)" as lgbm
participant "Catálogo\nen Memoria" as catalog

user -> browser : Selecciona 8 cartas
browser -> api : POST /predict-deck\n{ cardIds: [...] }
api -> catalog : Consultar cartas\npor ID
catalog --> api : Datos de cartas\n(elixir, arquetipos, stats)
api -> api : Calcular costo promedio\nCalcular sinergia\nDistribución arquetipos
api -> lgbm : Inferencia\n(features vector)
lgbm --> api : Probabilidad victoria\n(0.0 - 1.0)
api -> api : Calcular meta rating
api --> browser : JSON\n{ metrics, wrate, synergy }
browser --> user : Renderizar dashboard\ncon métricas
@enduml
```

Figura 9.2: Diagrama de secuencia del flujo de predicción de rendimiento de mazo en tiempo real.

Fuente: Elaboración propia.

La Figura 9.2 detalla la ejecución sincrónica de una consulta de predicción. Destaca el papel de ONNX Runtime para resolver el vector de características de 8 cartas en milisegundos, retornando al usuario la probabilidad predicha de victoria y recomendaciones tácticas personalizadas.

9.2.3. Flujo de Datos: Simulación de Parches

La Figura 9.3 describe el flujo de datos cuando un usuario configura un parche de balance hipotético. El endpoint `POST /simulate-patch` aplica efectos directos sobre las cartas ajustadas, identifica cartas indirectamente afectadas por compartir arquetipos, ejecuta la red LSTM para proyectar las tasas de uso futuras tras el parche y activa el motor de simulación Monte Carlo con 10,000 iteraciones para generar escenarios alternativos del metajuego.

```
@startuml
!theme plain
skinparam sequenceMessageAlign center

actor "Usuario" as user
participant "Navegador\n(React SPA)" as browser
participant "Express.js\nAPI REST" as api
participant "ONNX Runtime\n(LSTM)" as lstm
participant "Motor\nMonte Carlo" as mc

user -> browser : Configura buffs/nerfs\npor carta
browser -> api : POST /simulate-patch\n{ adjustments: [...] }
api -> api : Aplicar efectos directos\nIdentificar efectos indirectos
api -> lstm : Proyectar uso futuro\n(ventana 1,000 batallas)
lstm --> api : Tasas de uso predichas\npróxima temporada
api -> mc : Ejecutar 10,000\nsimulaciones
mc --> api : Distribución resultados\n(colapso/auge arquetipos)
```

```
api --> browser : JSON\n{ simulatedCards, deduction }  
browser --> user : Renderizar gráfica\ncomparativa  
@enduml
```

Figura 9.3: Diagrama de secuencia del flujo de simulación de parches de balance y proyección Monte Carlo.

Fuente: Elaboración propia.

La Figura 9.3 refleja la interacción multimodelo durante la simulación de parches. Se acopla la proyección secuencial de la LSTM con el muestreo estocástico de Monte Carlo, permitiendo prever la estabilidad y variabilidad de los arquetipos en la siguiente temporada competitiva.

9.3. Justificación de Tecnologías

9.3.1. Frontend: React + TypeScript

Se seleccionó **React 19** por su ecosistema maduro de componentes, su modelo de virtual DOM para actualizaciones eficientes y la amplia disponibilidad de librerías para visualización de datos. **TypeScript** fue incorporado para garantizar tipado estático y detección de errores en tiempo de compilación. **Tailwind CSS** se eligió sobre Bootstrap o CSS puro por su enfoque de utilidades atómicas que permiten diseño responsivo sin JavaScript adicional.

9.3.2. Backend: Express.js + ONNX Runtime

Express.js fue seleccionado por su minimalismo y compatibilidad con el ecosistema Node.js, lo que permite desplegar tanto el frontend como el backend en una misma plataforma (Vercel). Para la inferencia de los modelos de machine learning se utilizó **ONNX Runtime** (ONNX Runtime developers, 2021), que permite ejecutar modelos entrenados en PyTorch y scikit-learn directamente en Node.js sin necesidad de un microservicio Python separado.

9.3.3. Despliegue: Vercel

Vercel se eligió por su integración nativa con React y soporte para funciones serverless, lo que elimina la necesidad de gestionar infraestructura de servidores. El archivo `vercel.json` configura reescrituras de rutas para dirigir las peticiones API hacia la función serverless correspondiente.

9.4. Pipeline de Entrenamiento y Exportación de Modelos

El proceso de integración de los modelos predictivos en el aplicativo web requiere resolver una brecha técnica fundamental: los modelos se entrenan en **Python** utilizando PyTorch y scikit-learn, pero el servidor de inferencia opera en **Node.js** (TypeScript). Para resolver esta incompatibilidad se adoptó el formato **ONNX** (Open Neural Network Exchange) (Ke et al., 2017) como formato intermedio de serialización. La Figura 9.4 describe el flujo completo desde el entrenamiento en Python hasta la inferencia en el navegador.

```
@startuml
!theme plain
skinparam sequenceMessageAlign center

participant "Datos\nHistóricos" as data
participant "Python\n(Entrenamiento)" as python
participant "Archivos\nONNX" as onnx
participant "Node.js\n(Inferencia)" as node
participant "Navegador\n(React SPA)" as browser

data -> python : CSV series temporales\nCSV registros sintéticos
python -> python : Entrenar LSTM\n(PyTorch)
python -> python : Entrenar LightGBM\n(scikit-learn)
python -> onnx : torch.onnx.export\n(opset 17)
python -> onnx : onnxmltools\n.convert_lightgbm\n(opset 12)
```

```

onnx -> node : Carga al iniciar\n(ort.InferenceSession.create)
node -> browser : API REST\npredicciones
@enduml

```

Figura 9.4: Diagrama del pipeline de entrenamiento en Python y exportación a ONNX para inferencia en Node.js.

Fuente: Elaboración propia.

La Figura 9.4 ilustra el pipeline de conversión. Los modelos se entrenan en Python, se convierten a grafos serializados .onnx mediante `torch.onnx.export` y `onnxmltools`, y finalmente son cargados en memoria por Node.js al arrancar el servidor REST.

9.5. Estructura y Módulos del Aplicativo

9.5.1. Componentes de la Interfaz (Frontend)

Componente	Responsabilidad
<code>App.tsx</code>	Navegación principal, layout y estado global
<code>DeckPredictor.tsx</code>	Construcción de mazos y predicción de rendimiento
<code>PatchSimulator.tsx</code>	Simulación de cambios de balance
<code>HistoryTrends.tsx</code>	Visualización temporal de tendencias
<code>ModelWorkbench.tsx</code>	Comparación de métricas de modelos ML

Fuente: Elaboración propia para la arquitectura de MetaPredict.

9.5.2. Controlador API REST (Backend)

Método	Ruta	Función
GET	/api/clash/cards	Retorna el catálogo completo de 26 cartas
POST	/api/clash/predict-deck	Evalúa un mazo de 8 cartas y retorna métricas
POST	/api/clash/simulate-patch	Simula el impacto de un parche de balance
GET	/api/clash/model-metrics	Retorna métricas de rendimiento de los modelos

Fuente: Especificación técnica de la API REST MetaPredict (Elaboración propia).

9.6. Métricas de Rendimiento

Se evaluó el rendimiento del sistema en un entorno de pruebas con las siguientes especificaciones: Intel Core i5-12400, 16 GB RAM, Node.js v20.

Operación	Tiempo Promedio	Desviación Estándar
Carga inicial del servidor	1.2 s	0.3 s
Predicción de mazo (LightGBM)	12 ms	2 ms
Proyección temporal (LSTM)	45 ms	8 ms
Simulación Monte Carlo (10K iteraciones)	180 ms	25 ms
Respuesta completa /predict-deck	65 ms	12 ms

Fuente: Pruebas de rendimiento ejecutadas localmente en entorno de laboratorio (Elaboración propia).

El tiempo de respuesta total para la predicción de un mazo es de aproximadamente **65 milisegundos**, lo que permite una experiencia interactiva en tiempo real para el usuario.

9.7. Despliegue y Escalabilidad

El aplicativo se despliega en **Vercel** utilizando funciones serverless. El archivo `vercel.json` configura reescrituras de rutas para dirigir todas las peticiones `/api/*` hacia la función serverless que ejecuta la aplicación Express.

En entorno de desarrollo, **Vite** se ejecuta como middleware de Express, proporcionando *hot module replacement* (HMR) para un ciclo de desarrollo ágil. En producción, los archivos estáticos del frontend se sirven directamente desde el CDN de Vercel, garantizando tiempos de carga mínimos y alta disponibilidad del servicio.

Capítulo 10

Conclusiones e Investigación de Campo

A partir de la experimentación computacional realizada utilizando 50,000 registros históricos y el entrenamiento de 6 modelos predictivos sobre CPU, se establecen las siguientes conclusiones científicas:

1. **Predominio Predictivo Recurrente (LSTM):** Se valida la hipótesis de investigación: los modelos de aprendizaje automático estructurados para series de tiempo permiten predecir las tendencias futuras del metajuego de Clash Royale. La red LSTM obtuvo la mayor precisión del estudio con un MAE de **0.006103**, superando a Prophet y ensambles estáticos gracias a su capacidad de retener dependencias dinámicas de largo plazo.
2. **Jerarquía del Éxito Competitivo (Nivel Micro):** Mediante la importancia de variables calculada con Random Forest, se determinó que la *diferencia de nivel acumulado de las cartas* (0.1374) y la *habilidad acumulada / copas del jugador* (0.1899) son los factores más determinantes para la victoria en combates individuales. Esto indica que el nivel físico de las unidades y el rango del jugador dominan sobre la mera composición genérica del mazo.
3. **Validación de Balance de Rarezas:** El análisis micro descriptivo demostró que las barajas ganadoras y perdedoras poseen distribuciones de rarezas estadísticamente idénticas (promedio de 1.6 legendarias por mazo). Esto confirma empíricamente la hipótesis de balance de Supercell: poseer más cartas legendarias o épicas en el mazo **no aumenta la probabilidad de victoria**.

Referencias

- Breiman, L. (2001). *Random Forests*. Machine Learning, 45(1), 5-32. <https://link.springer.com/article/10.1023/A:1010933404324>
- Cerdá Tena, E., Pérez Navarro, J., & Jimeno Pastor, J. L. (2004). *Teoría de Juegos*. Pearson Educación, S.A.
- Dou, M. (2024). *Principle and Applications of Monte-Carlo Simulation in Forecasting*. Highlights in Science, Engineering and Technology, Vol. 88, pp. 406-414. <https://www.grafiati.com/en/literature-selections/monte-carlo-simulation-method>
- Facebook/Meta Prophet contributors. (2025). *Quick Start – Prophet*. Prophet Official Documentation. https://facebook.github.io/prophet/docs/quick_start.html
- Fernández Genaro, J. L. (2023). *Modelos de Machine Learning para la Ciencia de Datos* [Trabajo de Fin de Grado]. Universidad de Salamanca, Facultad de Ciencias, Departamento de Estadística.
- Hyndman, R. J., & Athanasopoulos, G. (2021). *Forecasting: Principles and Practice* (3rd ed.). OTexts. <https://otexts.com/fpp3/>
- Ke, G., Li, Q. M., Finley, T., Odintsev, A., Li, X., Shen, Z., & Wu, T.-Y. (2017). *ONNX: Open Standard for ML Interoperability*. <https://onnx.ai/onnx/>
- LightGBM contributors. (2025). *LightGBM Documentation*. Read the Docs. <https://lightgbm.readthedocs.io/en/latest/>
- MetricGate. (2026). *Time-Series Walk-Forward (Rolling-Origin) Cross-Validation*. MetricGate Documentation. <https://metricgate.com/docs/time-series-walk-forward-cv>
- ONNX Runtime developers. (2021). *ONNX Runtime*. <https://onnxruntime.ai/>
- PyTorch contributors. (2025). *torch.nn – PyTorch Documentation*. PyTorch Official Documentation.

<https://pytorch.org/docs/stable/nn.html>

Russell, R. (2018). *Machine Learning: Guía Paso a Paso Para Implementar Algoritmos De Machine Learning Con Python*. Rudolph Russell.

Scikit-learn contributors. (2025). 3.4. *Metrics and Scoring: Quantifying the Quality of Predictions*. scikit-learn 1.9.0 Documentation. https://scikit-learn.org/stable/modules/model_evaluation.html

Tam, A. (2023). *LSTM for Time Series Prediction in PyTorch*. Machine Learning Mastery. <https://machinelearningmastery.com/lstm-for-time-series-prediction/>

XGBoost contributors. (2025). *XGBoost Documentation*. XGBoost Official Documentation. <https://xgboost.readthedocs.io/en/latest/>